Input and Output

27 April 2026 11:05

Sometimes a program needs to interact with the users to get some input data or information from the end user and process it to give the desired output. In Python we have the input () function for taking the user input. The input () function prompts the user to enter data. It accepts all user input as string. The user may enter a number of string but the input () function treats them as string only. The syntax for input() is:

```
input([prompt])
```

Prompt is the string that we may like to display on the screen prior to taking the input and it is optional. When a prompt is specified first it is displayed on the screen, after which the user can enter the data. The input () Takes exactly what is typed from the keyboard, converts it into a string and assign it to the variable on the left hand side of the assignment operator (=). Entering data for the input function is terminated by pressing the Enter key.

```
fname = input('Enter your first name: ')
Enter your first name: Shiv
age = input('Enter your age: ')
Enter your age: 17
type(age)
<class 'str'>

age = int(input('Enter your age: '))
Enter your age: 18
type(age)
<class 'int'>
```

Here 'age' is in int data type.

Python uses the **print()** function to output data to standard output device - the screen. The function print () evaluates the expression before displaying it on the screen. The print () outputs a complete line and then moves to the next line for a subsequent output. The syntax for print () is:

```
print(value[,…, sep = ' ', end = '\n'])
```

- **sep:** The optional parameter **sep** is a separator between the output values. We can use the character integer or a string as a separator. The default separator is space.
  For example:

```
fname
'Shiv'
age
18
st
'abcdecbac dba kcsb'

print(fname, age, st)
Shiv 18 abcdecbac dba kcsb
print(fname, age, st, sep='$')
Shiv$18$abcdecbac dba kcsb
```

sep default is space

seperator between two content.

```
print(fname, age, st, sep='gamma')
Shivgamma18gammaabcdecbac dba kcsb
print(fname, age, st, sep='#')
Shiv#18#abcdecbac dba kcsb
```

- **end:** This is also optional and it allows us to specify any string to be appended after the last value. The default is a new line. For example:

```
fname = 'Shiv'
lname = 'Ray'
print(fname, end=' ')
print(lname)
```

Type Conversion

```
a = 18
b = 22
a + b
40
r = a + b
r
40
type(r)
<class 'int'>
r = a / 4
r
4.5
type(r)
<class 'float'>
c = 'ten'
r = a + c
Traceback (most recent call last):
  File "<pyshell#46>", line 1, in <module>
    r = a + c
TypeError: unsupported operand type(s) for +: 'int' and 'str'
r = c + a
Traceback (most recent call last):
  File "<pyshell#47>", line 1, in <module>
    r = c + a
TypeError: can only concatenate str (not "int") to str
```

implicit type conversion done by Python.

Python is unable to do conversion.

Python is unable to do conversion.

Explicit conversion: When the conversion is done by the programmer's consent then it is called explicit conversion.

For example:

```
c
'ten'
a
18
type(c)
<class 'str'>
type(a)
<class 'int'>
```

Explicit conversion is done here

```
<class 'str'>
type(a)
<class 'int'>

r = c + str(a)
r
'ten18'
```

explicit conversion is done here.

int

It down, explicit conversion, also called type casting, happens when data type conversion takes place because the programmer forced it in the program. The general form of an explicit data type conversion is:

(new_data_type)(expression)

With explicit type conversion, there is a risk of loss of information since we are forcing an expression to be of a specific type. For example, converting a floating value of X equal to 20.67 into an integer type that is. In the (X) Will discard the fractional part 0.67. Following are some of the function in Python that are used for explicitly converting an expression or a variable to a different type.

Function	Description
int(x)	Converts x to an integer.
float(x)	Convert x to a floating point number.
str(x)	Convert x to a string representation.
boolean(x)	Convert x to a boolean representation.
complex(x)	Convert x to a complex representation.

Homework:

1. Basic Input & Output
Write a program to input your **name, age, and city** and display them in the following format:
Name: ___
Age: ___
City: ___

2. Type Identification
Write a program that takes two inputs:
- one integer
- one string

Print both values and display their **data types using type().**

3. String to Integer Conversion
Take a number as input using input() and:
- Convert it into an integer
- Print its square

4. Separator Usage
Write a program to input three values: **name, marks, grade** and print them using:
- default separator
- # as separator
- *** as separator

5. End Parameter

Write a program to print:
Hello World
Welcome to Python
on the **same line** using the end parameter.

6. Arithmetic with Input
Take two numbers as input from the user and display:
- Sum
- Difference
- Product
- Division

Ensure proper type conversion.

7. Implicit Type Conversion
Write a program:
- Assign integer value to a variable
- Divide it by another number
- Print the result and its type
  Explain why the type changed.

8. Error Handling Concept
Write a program where:
- a = 10
- b = "5"

Try:
print(a + b)
Then fix the error using proper type conversion.

9. String Concatenation with Numbers
Take a number as input and concatenate it with the string "Number entered is: "
Example output:
Number entered is: 25

10. Mixed Conversion Problem
Write a program that:
- Takes **length and breadth** as input (as strings)
- Converts them into integers
- Calculates and prints the **area and perimeter**

Debugging

A programmer can make mistakes while writing a program, and hence the program may not execute or may generate wrong output. The process of identifying and removing such mistakes, also known as bugs or errors from the program, is called debugging.
Errors occurring in programs can be categorized as:

i. Syntax errors
ii. Logical errors
iii. Runtime errors

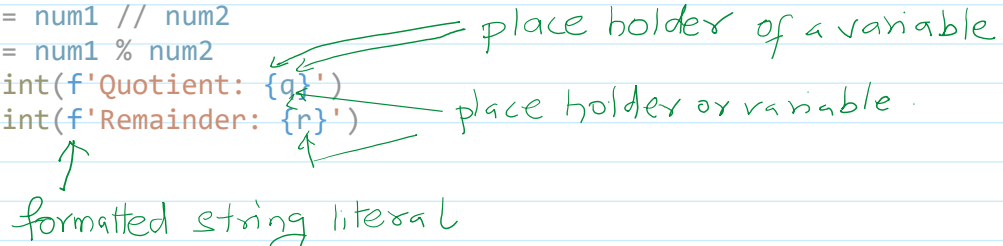
1. **Syntax errors:** Like other programming languages, Python has its own rules that determine its syntax. The interpreter interprets the statement only if it is syntactically (As per the rules of Python) is correct. If any syntax error is present, the interpreter shows the error messages and stops the execution there. For example. Parenthesis must be in pairs, so the expression (10+12) Is

syntactically correct, whereas (7 + 11 is not due to absence of right parentheses. Such errors need to be removed before the execution of the program.

2. **Logical Errors:** A logical error is a bug in the program that causes it to behave incorrectly. A logical error produces an undesired output, but without abrupt termination of the execution of the program. Since the program interprets successfully even with logical errors are present in it, it is sometimes difficult to identify these errors. The only evidence to existence of logical errors is the wrong output. While working backwards from the output of the program one can identify what went wrong.
   Say for example if we wish to find the average of two numbers 10 and 12 and we write the code as 10 + 12/2 it would run successfully and produce the result 16. Surely 16 is not the average of 10 and 12. The correct code to find the average should have been (10 + 12) divide by two to give the correct output as 11.
   Logical errors are also called semantic errors as they occur when the meaning of the program (Its semantics) is not correct.

3. **Runtime Error:** A runtime error causes abnormal termination of program while it is executing. Runtime error is when the statement is correct syntactically but the interpreter cannot execute it. Runtime errors do not appear until after the program starts running or executing.
   For example, we have a statement having division operation in the program. By mistake if the denominator entered is zero then it will give a runtime error like **"division by 0".**

```
num1 = int(input('Enter dividend: '))
num2 = int(input('Enter divisor: '))
q = num1 // num2
r = num1 % num2
print(f'Quotient: {q}')
print(f'Remainder: {r}')
```



A concise and efficient way to embed expressions and variables directly into string literals in Python. Introduced in Python's 3.6 version, it is currently the preferred and the fastest method of string formatting.

```
num1 = int(input('Enter dividend: '))
num2 = int(input('Enter divisor: '))
q = num1 // num2
r = num1 % num2
div = num1 / num2
print(f'Quotient: {q}')
print(f'Remainder: {r}')
print(f'Divide: {div:.2f}')
```

Outputs:

```
Enter dividend: 20
Enter divisor: 7
Quotient: 2
Remainder: 6
Divide: 2.86
```

Runtime Error:

```
Enter dividend: 45
Enter divisor: 0
Traceback (most recent call last):
  File "C:\Users\bluej\Documents\Students\ShivRay\ForGit\Coding\zero\divide.py", line 4, in <module>
    q = num1 // num2
        ~~~~~^^~~~~
ZeroDivisionError: division by zero
```

```
Enter dividend: twenty
Traceback (most recent call last):
  File "C:\Users\bluej\Documents\Students\ShivRay\ForGit\Coding\zero\divide.py", line 1, in <module>
    num1 = int(input('Enter dividend: '))
ValueError: invalid literal for int() with base 10: 'twenty'
```

```
Enter dividend: 34.6
Traceback (most recent call last):
  File "C:\Users\bluej\Documents\Students\ShivRay\ForGit\Coding\zero\divide.py", line 1, in <module>
    num1 = int(input('Enter dividend: '))
ValueError: invalid literal for int() with base 10: '34.6'
```

f-string

To create an F - string, prefix the string with f or F and use curly braces {} to include variables or expression.

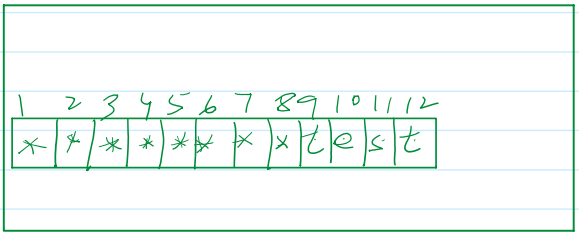
```
num1 = int(input('Enter dividend: '))
num2 = int(input('Enter divisor: '))
q = num1 // num2
r = num1 % num2
print(f'Quotient: {q}')
print(f'Remainder: {r}')
```

```
print(f'{'test':*>12}')
```

number of spaces blocked for each character

right justified

vaccant spaces filled by this character.




```
print(f'{'test':*>12}')
print(f'{'test':*<12}')
print(f'{'test':*^12}')
```

right justified

left justified.

centre.

padding and alignment

Home work

Home work

1. Student Information Formatter
Write a program to input the following details of a student:
- Name
- Class
- Section
- Roll Number
- Percentage

Display the output in a well-formatted manner using:
- sep
- end
- f-strings

Also display the percentage rounded to 2 decimal places.

2. Electricity Bill Generator
Write a program to input:
- Customer name
- Previous meter reading
- Current meter reading
- Cost per unit

Calculate:
- Units consumed
- Total bill amount

Display the bill using formatted strings and proper alignment.

3. Shopping Invoice Program
Write a program that inputs:
- Product name
- Quantity
- Price per item

Calculate:
- Total amount
- GST (18%)
- Final bill

Display the invoice using f-string formatting with:
- Right alignment
- Currency values up to 2 decimal places

4. Temperature Conversion System
Write a menu-driven program to:
1. Convert Celsius to Fahrenheit
2. Convert Fahrenheit to Celsius

Use explicit type conversion wherever necessary and display results using formatted output.

5. Salary Slip Generator
Write a program to input:
- Employee name
- Basic salary

Calculate:
- HRA = 20% of salary
- DA = 15% of salary

- PF = 12% deduction
- Net salary

Display the salary slip neatly using f-strings and alignment formatting.

6. Number Analyzer

Write a program to input a number as a string and then:

- Convert it into integer
- Find its square
- Find its cube
- Find whether it is even or odd

Display all results along with their data types.

7. Restaurant Billing System

Write a program that inputs:

- Customer name
- Food item
- Quantity
- Price per item

Calculate:

- Total bill
- Discount (10% if bill exceeds 1000)
- Final payable amount

Use:

- sep
- end
- f-string formatting

for attractive output.

8. Area and Perimeter Calculator

Write a program that inputs dimensions as strings and calculates:

1. Area and perimeter of rectangle
2. Area and circumference of circle
3. Area of triangle

Use proper explicit conversion and formatted display.

9. Marksheet with Grade Calculation

Write a program to input marks of 5 subjects.

Calculate:

- Total marks
- Percentage
- Grade according to percentage

Display the complete marksheet using f-strings with aligned formatting.

Grade rules:

- 90+ → A+
- 75–89 → A
- 60–74 → B
- 40–59 → C
- Below 40 → Fail

10. Banking Transaction Simulator

Write a program to input:

- Account holder name
- Initial balance
- Deposit amount

- Withdrawal amount

Calculate and display:

- Final balance

If withdrawal amount exceeds balance, display an error message.
Use proper debugging concepts and formatted output.

11. String and Number Operations

Write a program that takes:

- One integer input
- One floating-point input
- One string input

Perform the following:

- Add integer and float
- Concatenate string with integer using explicit conversion
- Display all variable types before and after conversion

12. Runtime Error Demonstration Program

Write a program that demonstrates runtime errors using:
1. Division by zero
2. Invalid integer conversion

Then modify the program to avoid the errors using proper input handling.

13. Patterned Output using f-Strings

Write a program to input a word and display it:

- Left aligned
- Right aligned
- Center aligned

using different fill characters such as:

- *
- #
- @

Width should be 20 characters.

14. Travel Expense Calculator

Write a program to input:

- Passenger name
- Distance travelled
- Cost per kilometer
- Number of passengers

Calculate:

- Total travel cost
- Cost per passenger

Display the result using formatted strings with 2 decimal precision.

15. Comprehensive Debugging Program

The following code contains syntax, logical and runtime errors:

```
num1 = input("Enter first number: ")
num2 = input("Enter second number: ")
avg = num1 + num2 / 2
print("Average = avg)
```

Tasks:
1. Identify the syntax error.
2. Identify the logical error.
3. Identify the runtime/type-related issue.

4. Rewrite the complete corrected program.
5. Display the average using an f-string rounded to 2 decimal places.